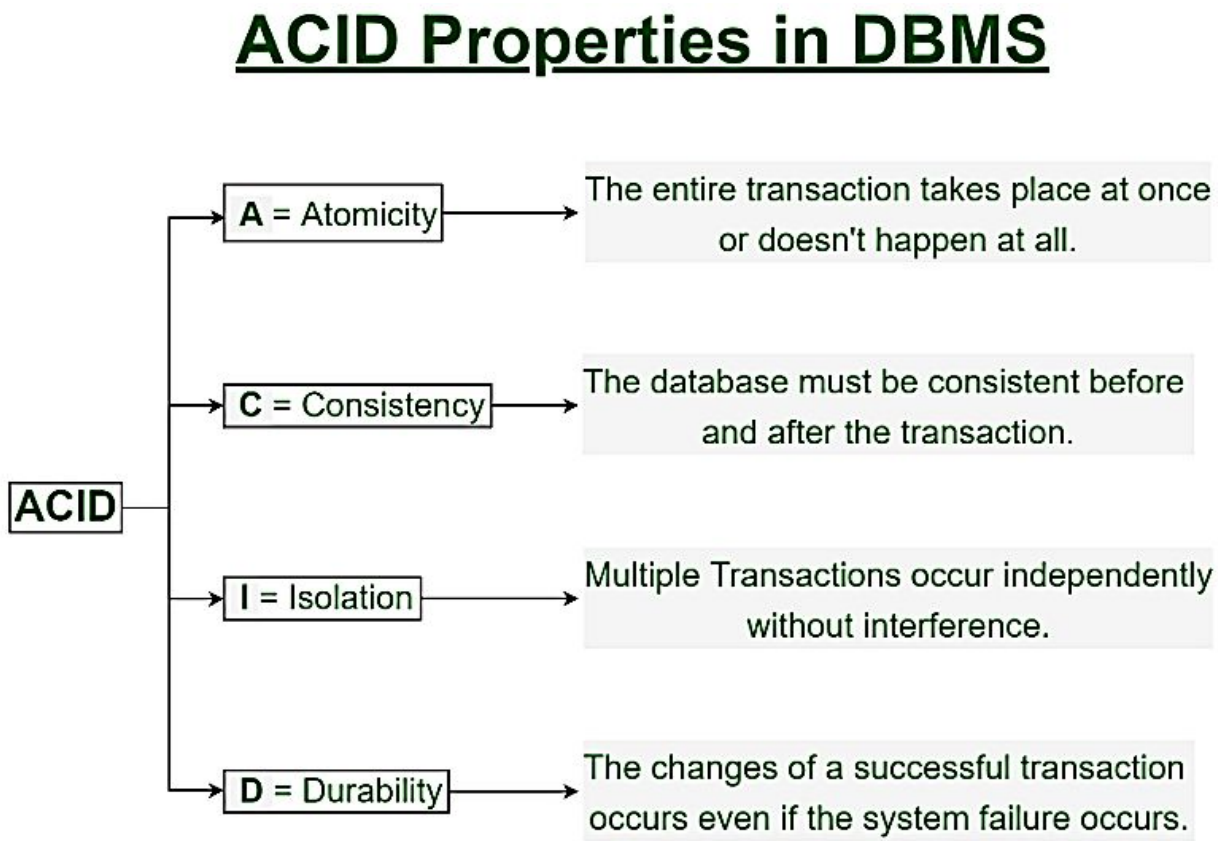
Property of Transaction (ACID)

1. Atomicity
2. Consistency
3. Isolation
4. Durability

Property of Transaction (ACID)



Introduction



Atomicity:

- By this, we mean that either the entire transaction takes place at once or doesn't happen at all. There is no midway i.e. transactions do not occur partially. Each transaction is considered as one unit and either runs to completion or is not executed at all. It involves the following two operations.
- —Abort: If a transaction aborts, changes made to the database are not visible.
- —Commit: If a transaction commits, changes made are visible.
- Atomicity is also known as the 'All or nothing rule'.
- Consider the following transaction T consisting of T1 and T2: Transfer of 100 from account X to account Y.

Atomicity:

Before: X : 500	Y: 200
Transaction T	
T1	**T2**
Read (X) X: = X − 100 Write (X)	Read (Y) Y: = Y + 100 Write (Y)
After: X : 400	Y : 300

- If the transaction fails after completion of T1 but before completion of T2.(say, after write(X) but before write(Y)), then the amount has been deducted from X but not added to Y. This results in an inconsistent database state. Therefore, the transaction must be executed in its entirety in order to ensure the correctness of the database state.

Consistency

- ✔ The integrity constraints are maintained so that the database is consistent before and after the transaction.
- ✔ The execution of a transaction will leave a database in either its prior stable state or anew stable state.
- ✔ The consistent property of database states that every transaction sees a consistent database instance.
- ✔ The transaction is used to transform the database from one consistent state to another consistent state.

Consistency:

- This means that integrity constraints must be maintained so that the database is consistent before and after the transaction. It refers to the correctness of a database. Referring to the example above,
- The total amount before and after the transaction must be maintained.
- Total before T occurs = 500 + 200 = 700.
- Total after T occurs = 400 + 300 = 700.
- Therefore, the database is consistent. Inconsistency occurs in case T1 completes but T2 fails. As a result, T is incomplete.

Isolation

- ✔ It shows that the data which is used at the time of execution of a transaction cannot be used by the second transaction until the first one is completed.
- ✔ In isolation, if the transaction T1 is being executed and using the data item X, then that data item can’t be accessed by any other transaction T2 until the transaction T1ends.
- ✔ The concurrency control subsystem of the DBMS enforced the isolation property

Isolation:

- This property ensures that multiple transactions can occur concurrently without leading to the inconsistency of the database state. Transactions occur independently without interference. Changes occurring in a particular transaction will not be visible to any other transaction until that particular change in that transaction is written to memory or has been committed. This property ensures that the execution of transactions concurrently will result in a state that is equivalent to a state achieved these were executed serially in some order.
- Let X= 500, Y = 500.

Isolation:

T	T''
Read (X) X: = X*100 Write (X) Read (Y) Y: = Y – 50 Write (Y)	Read (X) Read (Y) Z: = X + Y Write (Z)

- Suppose T has been executed till Read (Y) and then T'' starts. As a result, interleaving of operations takes place due to which T'' reads the correct value of X but the incorrect value of Y and sum computed by
- T'': (X+Y = 50, 000+500=50, 500)
- is thus not consistent with the sum at end of the transaction:
- T: (X+Y = 50, 000 + 450 = 50, 450).
- This results in database inconsistency, due to a loss of 50 units. Hence, transactions must take place in isolation and changes should be visible only after they have been made to the main memory.

Durability

- ✔ The durability property is used to indicate the performance of the database's consistent state. It states that the transaction made the permanent changes.
- ✔ They cannot be lost by the erroneous operation of a faulty transaction or by the system failure. When a transaction is completed, then the database reaches a state known as the consistent state. That consistent state cannot be lost, even in the event of a system's failure.
- ✔ The recovery subsystem of the DBMS has the responsibility of Durability property.

Durability:

- This property ensures that once the transaction has completed execution, the updates and modifications to the database are stored in and written to disk and they persist even if a system failure occurs. These updates now become permanent and are stored in non-volatile memory. The effects of the transaction, thus, are never lost.

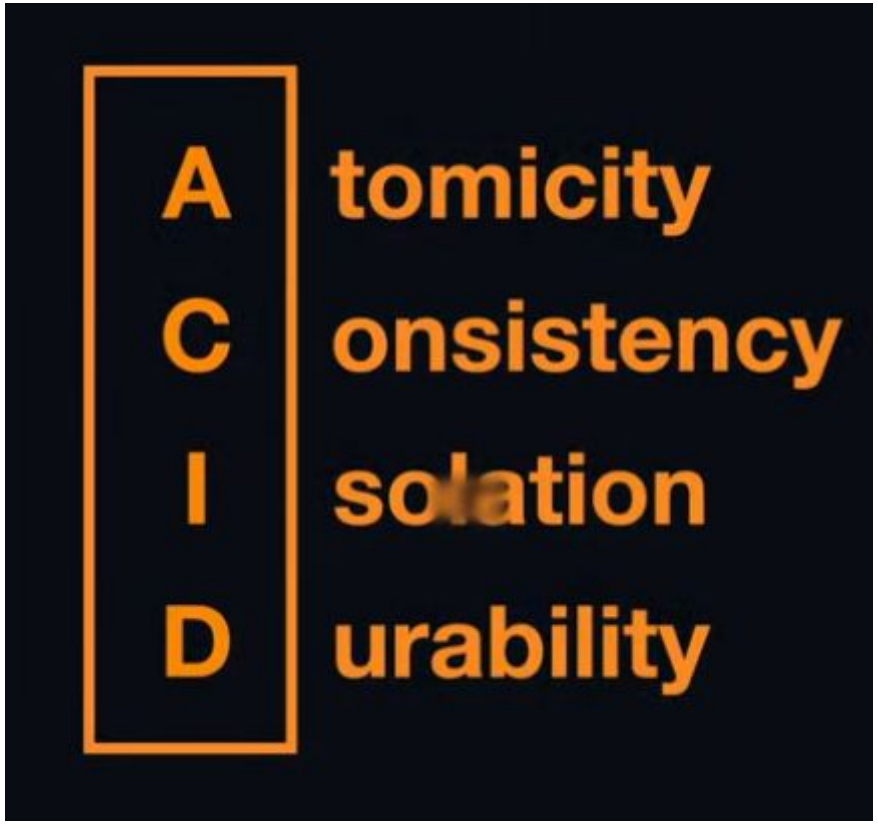
Some important points:

Property	Responsibility for maintaining properties
Atomicity	Transaction Manager
Consistency	Application programmer
Isolation	Concurrency Control Manager
Durability	Recovery Manager

Conclusion:

- The ACID properties, in totality, provide a mechanism to ensure the correctness and consistency of a database in a way such that each transaction is a group of operations that acts as a single unit, produces consistent results, acts in isolation from other operations, and updates that it makes are durably stored.


ACID


A tomicity
C onsistency
I solation
D urability

Atomicity

Entire transaction must finish.

Or revert the database to old state.

Single unit of Work = Multiple queries



Transfer $100 from Account 1 to Account 2



Transfer $100 from Account 1 to Account 2

1 — Check if Account 1 has $ 100 available

2 — Deduct $ 100 from account 1

3 — Add $ 100 to account 2



Transfer $100 from Account 1 to Account 2

- ✓ 1 — Check if Account 1 has $ 100 available
- 2 — Deduct $ 100 from account 1
- 3 — Add $ 100 to account 2



Account 1

Account 2

Transfer $100 from Account 1 to Account 2

- ✓ 1 — Check if Account 1 has $ 100 available
- ✓ 2 — Deduct $ 100 from account 1
- 3 — Add $ 100 to account 2



Transfer $100 from Account 1 to Account 2

- ✓ 1 — Check if Account 1 has $ 100 available
- ✓ 2 — Deduct $ 100 from account 1
- ✗ 3 — Add $ 100 to account 2



Account 1

Account 2

Transfer $100 from Account 1 to Account 2

- ✓ 1 — Check if Account 1 has $ 100 available
- ✓ 2 — Deduct $ 100 from account 1
- ✓ 3 — Add $ 100 to account 2

Consistency

Help ensure the data to be in Correct state.

Data integrity constraints must be followed

Constraints / Cascades / Triggers

Consistency = Correctness

Constraint — Amount must always be positive

Isolation

Concurrent execution safe.

Isolation

Concurrent execution safe.



Isolation

Concurrent execution safe.



$ 1000
$ 100

Total Amount $ 1100 - $ 1000 = - $100

Effects of a sequential transaction when running concurrent transactions.

Isolation

Concurrent execution safe.



Isolation

Concurrent execution safe.



Durability

Committed transactions to be persisted in non-volatile memory